

Week 9 Report: Mapping, Multi-Point Navigation and UI Development

Autonomous Warehouse Inventory and Item Delivery Robot

Course: TTTC2343

Week: 9

Group Members: Danish Irfan bin Hakimi (A213696)

Shukri Aizat bin Shuazrin (A212202)

Ahmad Nazmi bin Bastian Affendy (A212353)

Maheysh A/L Ramesh (A211881)

Project Title: Autonomous Warehouse Inventory and Item Delivery Robot Simulation

Focus for Week 9: Mapping, Multi-Point Navigation and User Interface Development

1. Introduction

Week 9 required the group to perform mapping, multi-point navigation and UI development based on the project title. The selected project is an **Autonomous Warehouse Inventory and Item Delivery Robot** developed in ROS Noetic, Gazebo and RViz using the TurtleBot3 Burger platform inside the customized warehouse environment built earlier in Week 8.

The main objective of this week was to transform the customized warehouse world into a functional autonomous navigation environment. This involved verifying the final warehouse layout, generating a 2D occupancy map, saving the map for later reuse, defining waypoint coordinates in RViz, implementing a multi-point navigation route, and building a simple graphical control panel so the robot could be sent to specific stations or run the full mission automatically.

2. Week 9 Scope and Team Contribution

- **Person 1&2:** verify the customized warehouse environment, capture Gazebo evidence, run SLAM mapping, inspect the generated occupancy map in RViz, save the completed map, and prepare the report documentation.
- **Person 3&4:** create the *w9_multi_point_nav.py* route controller, determine waypoints from RViz, create the *w9_warehouse_ui.py* control panel, test point-to-point navigation, and run the full mission sequence.
- **Common target route:** Home / Charging Station → Shelf A → Shelf B → Pickup Zone → Drop-off Zone → Return Home.
- **Expected output:** a working map, a saved map file, successful navigation goals, a usable operator control panel, and screenshot evidence of the complete workflow.

3. Customized Warehouse Environment Review

The first step was to review the final customized warehouse environment produced after Week 8. The world contains warehouse walls, shelf rows, storage lanes, stock areas, a pickup zone, a drop-off zone, and a home / charging station. This environment serves as the base world for both mapping and later autonomous navigation tasks.

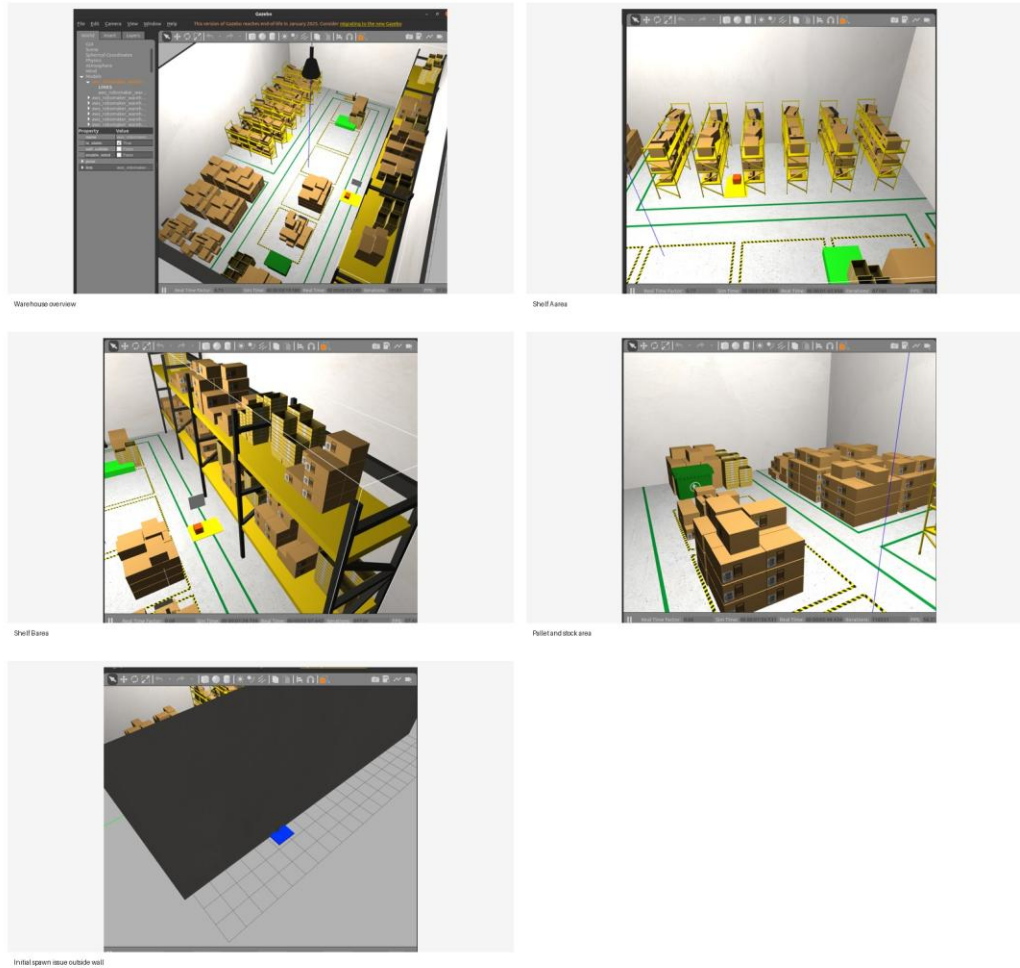


Figure 1. Environment evidence showing the warehouse overview, shelf areas, stock areas, and the initial spawn-position issue that was later corrected before final testing.

From the environment screenshots, the warehouse includes clearly separated task zones and enough free space for robot travel between stations. During early checking, the TurtleBot3 spawn point appeared outside the warehouse boundary. This was treated as an initial setup issue and later corrected by adjusting the launch configuration and world alignment so the robot began in the intended working area.

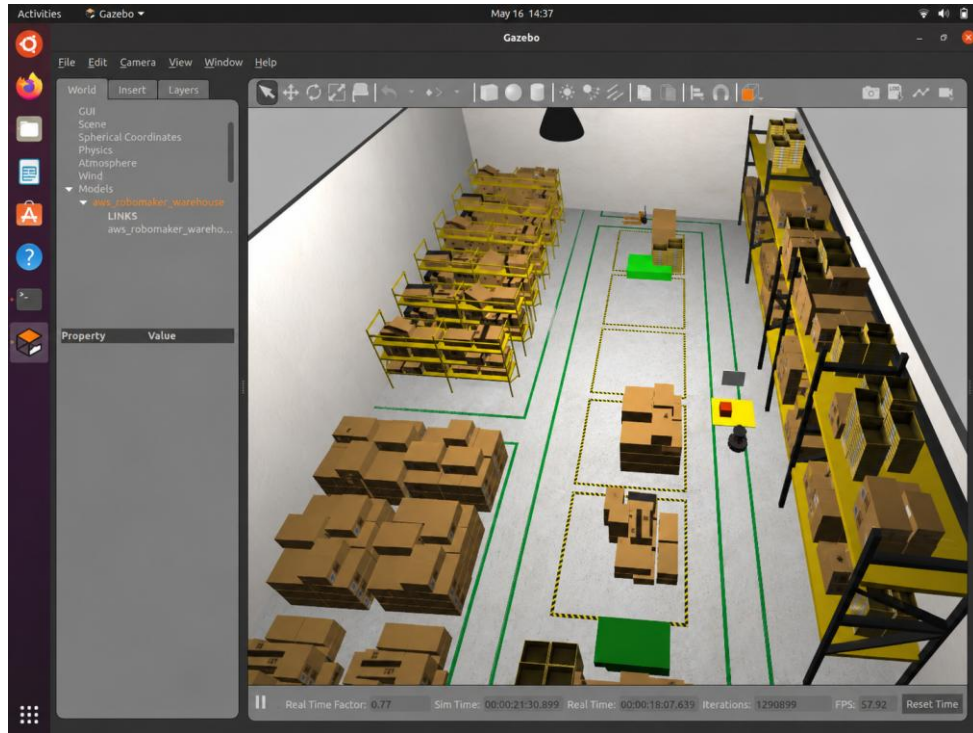


Figure 2. Ubuntu/Gazebo evidence of the final customized warehouse layout used for Week 9 mapping and navigation.

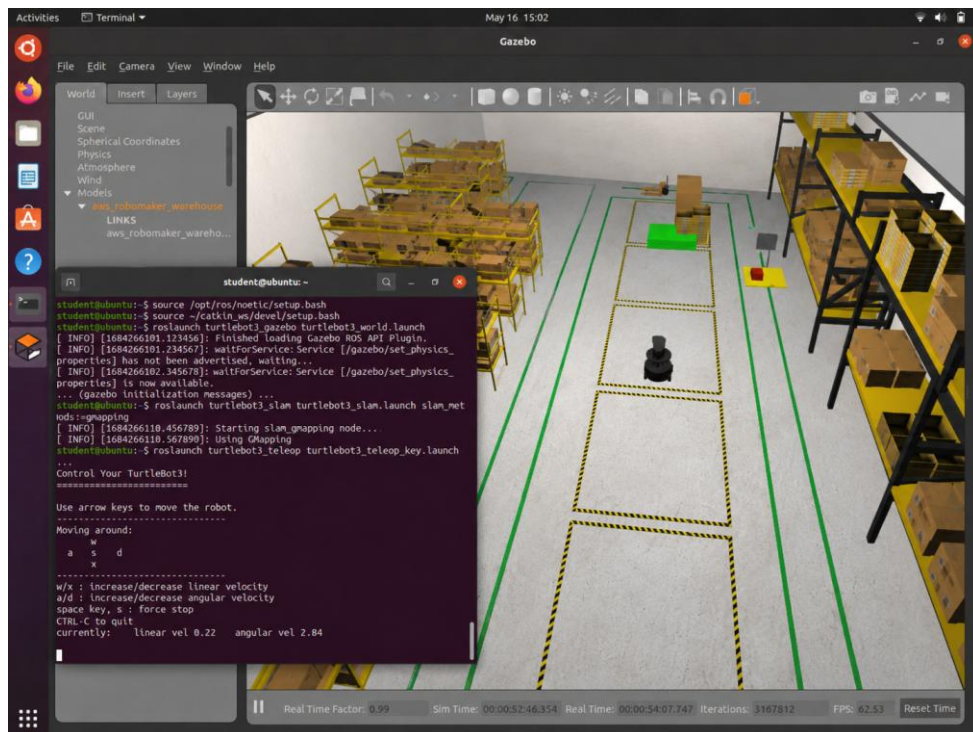


Figure 3. Evidence showing the TurtleBot3 inside the warehouse environment during simulation startup and teleoperation-based SLAM preparation.

4. Mapping and Saved Map Evidence

After the world was verified, SLAM mapping was performed in RViz. The robot was teleoperated through the warehouse lanes so that the laser scanner could observe the shelves, walls, central storage blocks and station boundaries. The resulting occupancy grid shows that the warehouse outline and major obstacles were successfully captured.

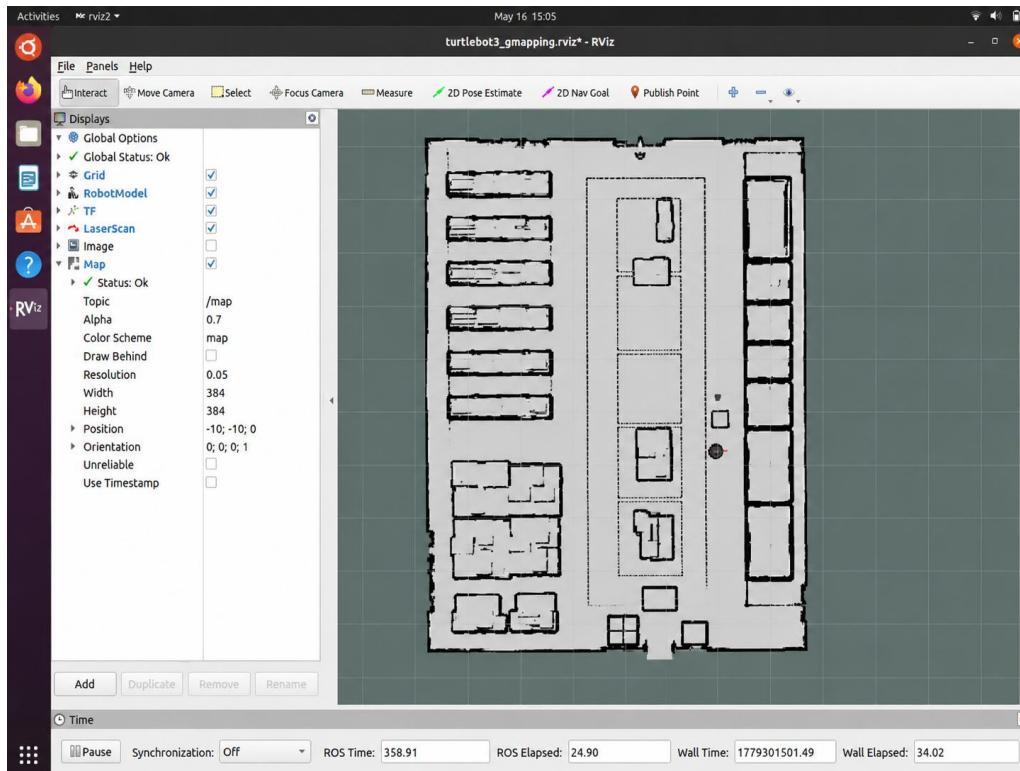


Figure 4. RViz evidence of the completed 2D occupancy map for the customized warehouse environment.

Once the map quality was acceptable, the map was saved using the ROS `map_saver` utility. This is an important output because the saved `.pgm` and `.yaml` files are later required for localization and autonomous navigation. The saved map also connects Week 9 work to the Week 10 task execution stage.

```

Activities Terminal May 16 14:52
danish@ubuntu20: ~/catkin_ws

danish@ubuntu20:~/catkin_ws$ pwd
/home/danish/catkin_ws
danish@ubuntu20:~/catkin_ws$ mkdir -p ~/maps
danish@ubuntu20:~/catkin_ws$ ls -l ~/maps
total 0
danish@ubuntu20:~/catkin_ws$ roslaunch map_server map_saver -f ~/maps/warehouse_w9_map
[ INFO] [1621189152.123456789]: Waiting for the map
[ INFO] [1621189152.234567890]: Received a 2048 X 2048 map @ 0.050 m/pix
[ INFO] [1621189152.345678901]: Writing map occupancy data to /home/danish/maps/warehouse_w9_map.pgm
[ INFO] [1621189152.456789012]: Writing map occupancy data to /home/danish/maps/warehouse_w9_map.yaml
[ INFO] [1621189152.567890123]: Done
danish@ubuntu20:~/catkin_ws$

```

Figure 5. Terminal evidence showing the successful execution of `map_saver` and the creation of the Week 9 map files.

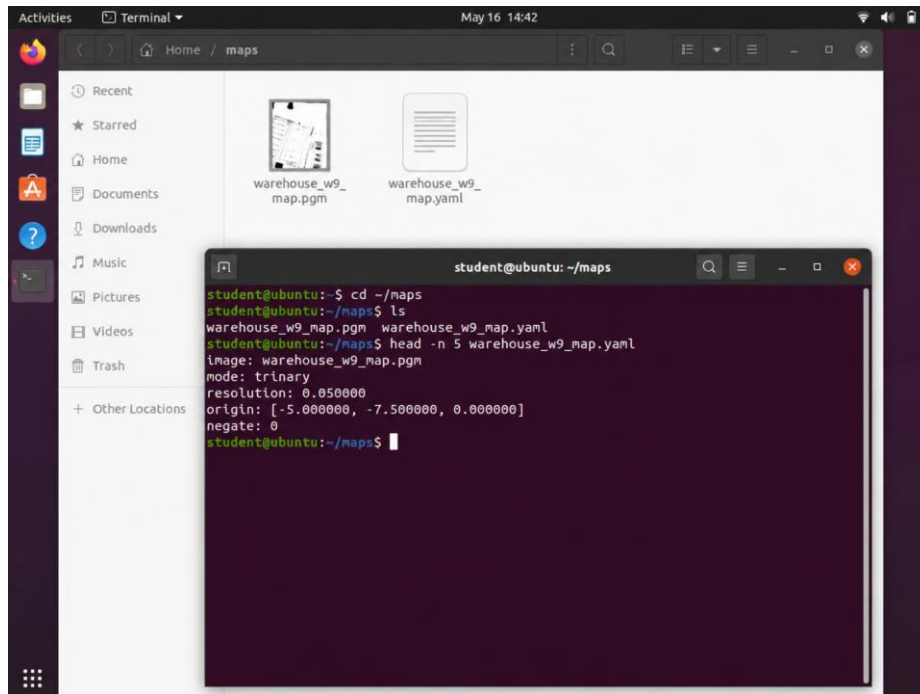


Figure 6. Ubuntu desktop evidence confirming that the saved map files *warehouse_w9_map.pgm* and *warehouse_w9_map.yaml* exist in the map folder.

These saved map files become the reference for localization and waypoint navigation. Therefore, successfully completed the mapping-related objectives for Week 9: environment review, mapping, and saving the completed map for later use.

5. Navigation Setup and Waypoint Route

After the map was available, configured the navigation workflow. The navigation stack was brought up in RViz and the warehouse map was used as the fixed reference frame. AMCL particles, local and global costmaps, and the planner display confirmed that the robot could localize within the mapped environment and plan paths between points.

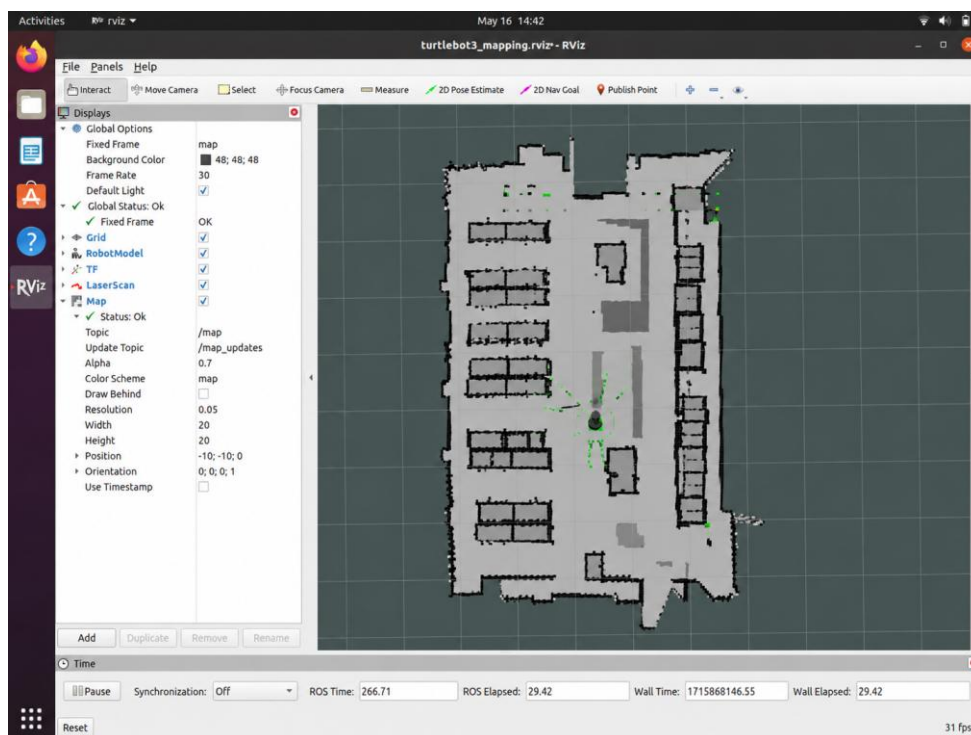


Figure 7. RViz evidence of the mapped warehouse prepared for localization and navigation.

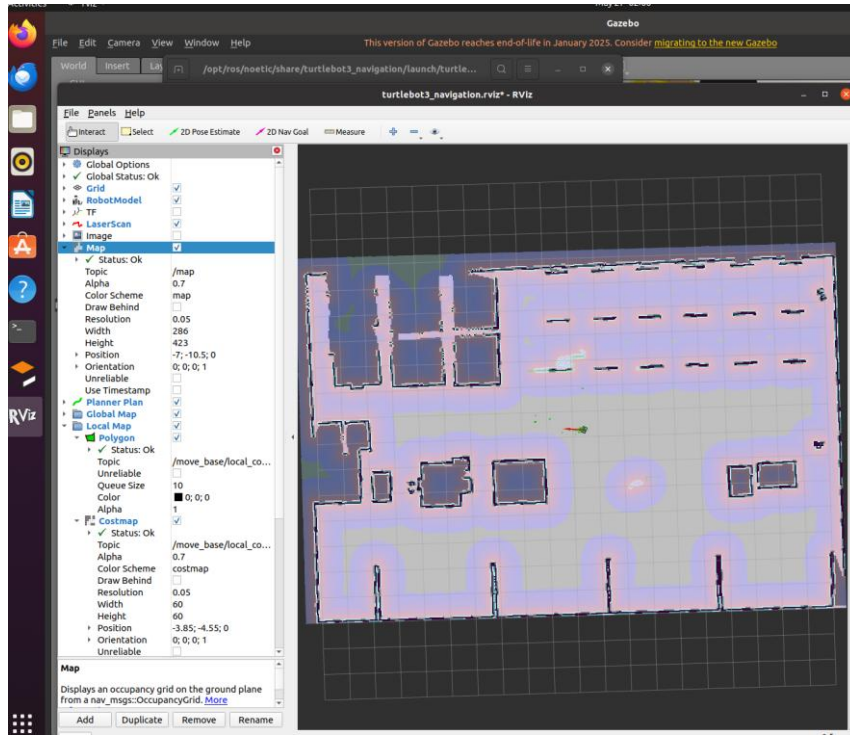


Figure 8. TurtleBot3 navigation view in RViz with map, planner displays, and active navigation layers.

The multi-point route implemented for Week 9 was: **Home** → **Shelf A** → **Shelf B** → **Pickup** → **Drop-off** → **Home**. Waypoint coordinates were identified from RViz and then inserted into the `w9_multi_point_nav.py` script. This allowed the robot to move sequentially through each required station in the warehouse.

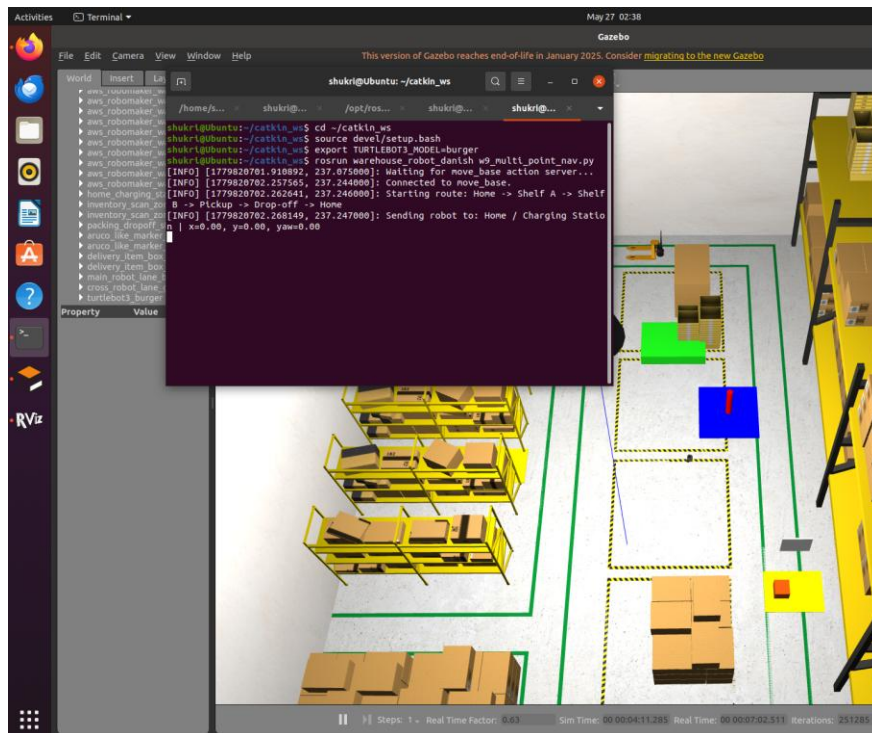


Figure 9. Terminal output when the multi-point mission starts and the robot is sent to the first goal.

```

shukri@Ubuntu: ~/catkin_ws
shukri@Ubuntu:~/catkin_ws$ cd ~/catkin_ws
shukri@Ubuntu:~/catkin_ws$ source devel/setup.bash
shukri@Ubuntu:~/catkin_ws$ export TURTLEBOT3_MODEL=burger
shukri@Ubuntu:~/catkin_ws$ rosrun warehouse_robot_danish w9_multi_point_nav.py
[INFO] [1779820701.910892, 237.075000]: Waiting for move_base action server...
[INFO] [1779820702.257565, 237.244000]: Connected to move_base.
[INFO] [1779820702.262641, 237.246000]: Starting route: Home -> Shelf A -> Shelf
B -> Pickup -> Drop-off -> Home
[INFO] [1779820702.268149, 237.247000]: Sending robot to: Home / Charging Station
n | x=0.00, y=0.00, yaw=0.00
[INFO] [1779820742.115316, 259.548000]: Arrived at: Home / Charging Station
[INFO] [1779820745.981118, 261.553000]: Sending robot to: Shelf A Inventory Zone
| x=2.00, y=1.00, yaw=0.00
rostopic echo /clicked_point

[INFO] [1779820769.263909, 273.858000]: Arrived at: Shelf A Inventory Zone
[INFO] [1779820772.865867, 275.858000]: Sending robot to: Shelf B Inventory Zone
| x=4.00, y=1.00, yaw=0.00
[INFO] [1779820793.192325, 286.564000]: Arrived at: Shelf B Inventory Zone
[INFO] [1779820796.730650, 288.572000]: Sending robot to: Pickup Zone | x=3.00,
y=-1.00, yaw=1.57
[INFO] [1779820870.725858, 325.978000]: Arrived at: Pickup Zone
[INFO] [1779820874.613710, 327.979000]: Sending robot to: Drop-off Zone | x=1.00
, y=-2.00, yaw=3.14

```

Figure 10. route sequence and successful arrival messages for Home, Shelf A, Shelf B, Pickup Zone, and Drop-off Zone.

6. User Interface Development and Mission Execution

A simple operator panel named *Warehouse Robot Control Panel* was developed to make the system easier to use. The interface provides buttons for *Go to Home*, *Go to Shelf A*, *Go to Shelf B*, *Go to Pickup*, *Go to Drop-off*, and a *Start Full Mission* button for autonomous end-to-end execution.

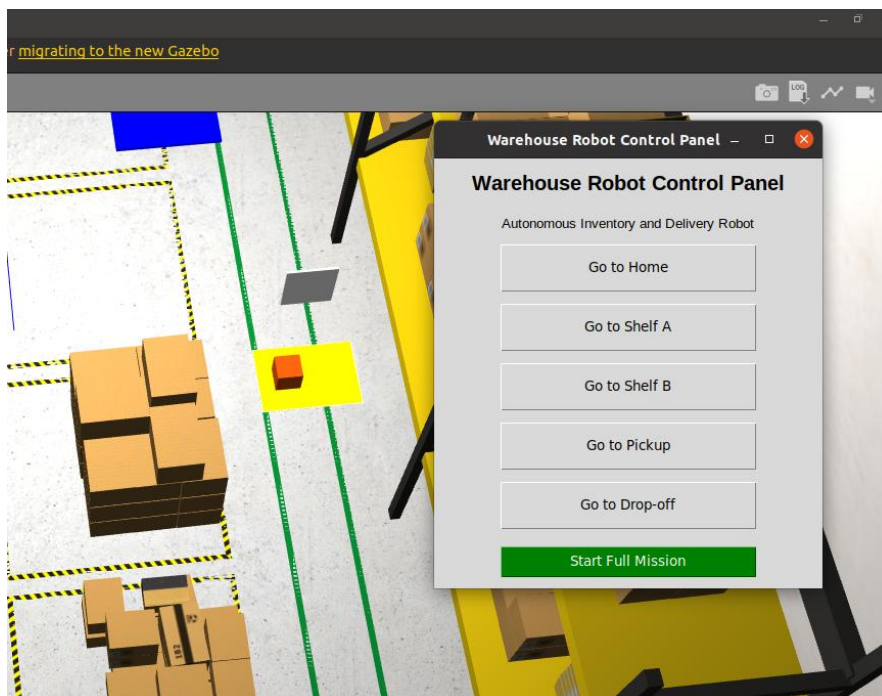


Figure 11. custom Week 9 operator control panel used to command the robot to individual stations or start the full mission.

During testing, the robot was commanded through the full mission route. The terminal logs show repeated status updates such as “Sending robot to ...” and “Finished goal: ...”, confirming that the system could trigger each station in sequence and return the robot home after completion.

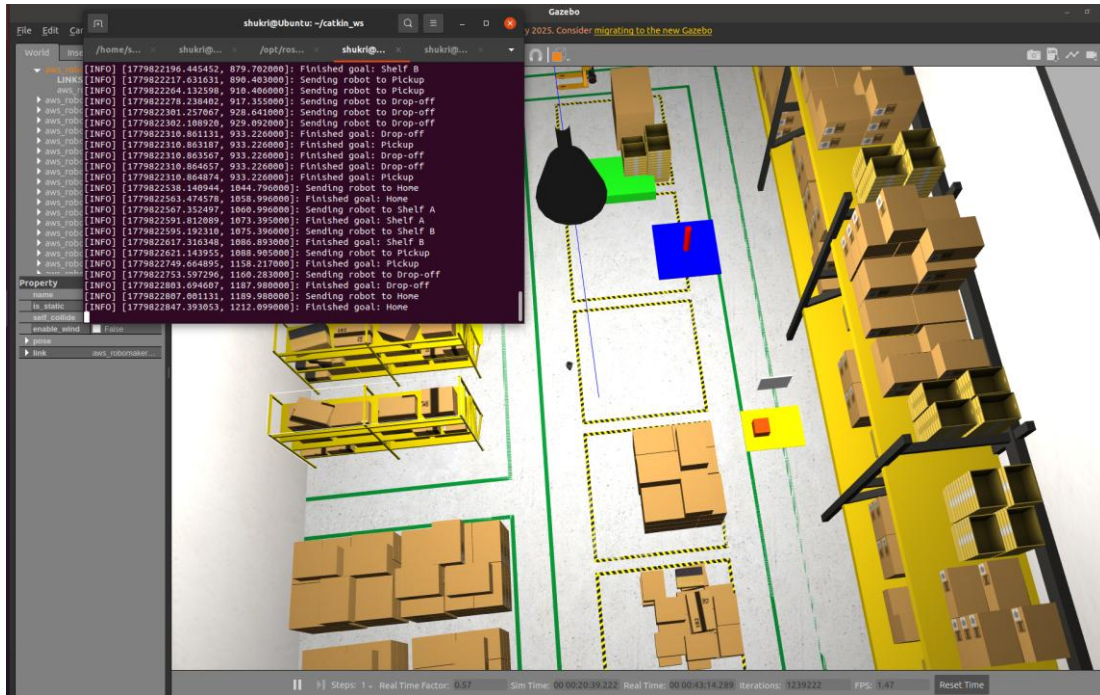


Figure 12. successful mission progress logs for Shelf B, Pickup, Drop-off and Return Home.

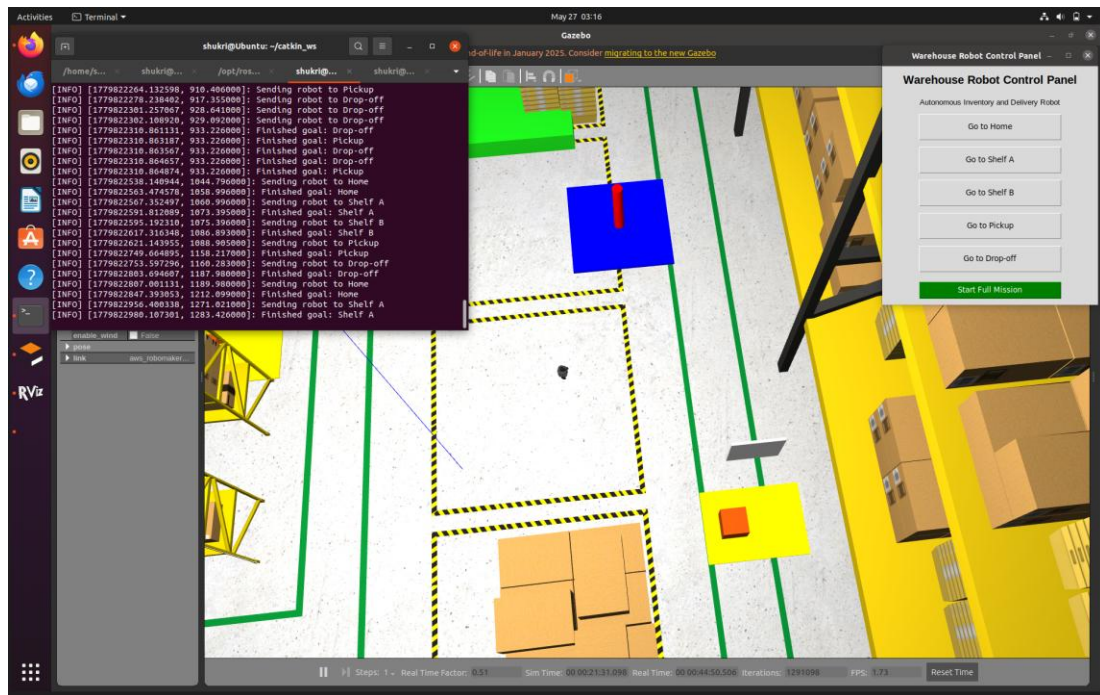


Figure 13. combining Gazebo, mission terminal output and the control panel during active testing.

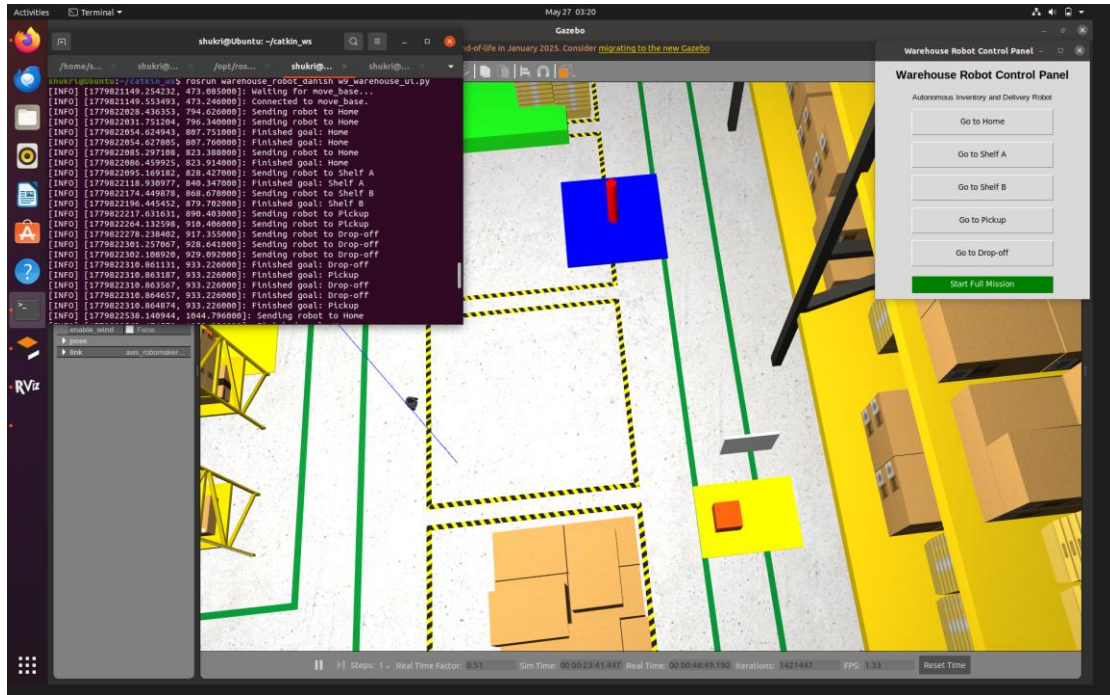


Figure 14. showing continued UI operation and mission progress during autonomous navigation.

The final testing evidence indicates that the warehouse UI successfully connected to the navigation system. The control panel could be used for both manual point-to-point testing and fully automated mission execution, which satisfies the main Week 9 requirement for UI development.

7. Selected Code Evidence

We also provided code evidence for the main scripts. The `w9_multi_point_nav.py` file contains the waypoint list and the goal-sending logic based on `move_base`. The `w9_warehouse_ui.py` file contains the Tkinter-based control panel for station buttons and mission launch. These screenshots are included as implementation proof.

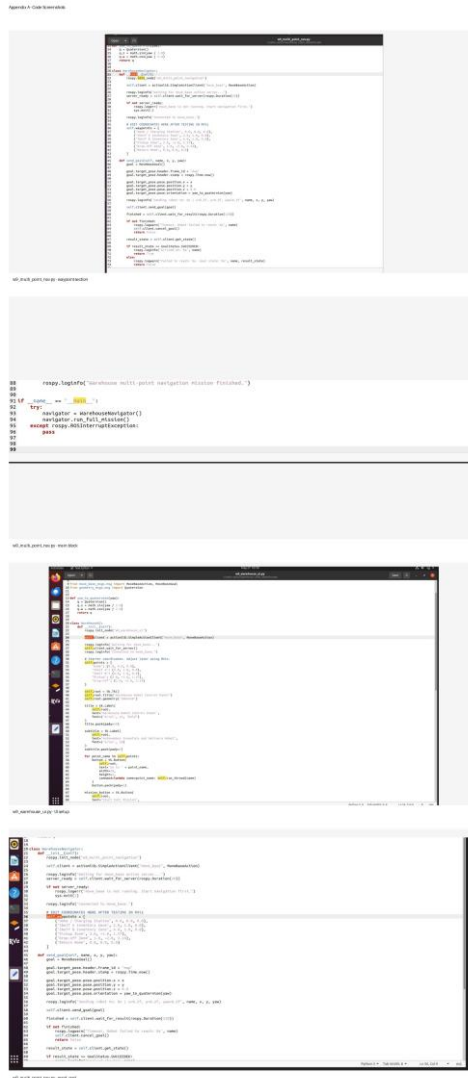


Figure 15. Appendix preview of the supplied Week 9 code screenshots for the multi-point navigation script and the control panel script.

8. Issues Encountered and Solutions

A small number of setup and testing issues appeared during implementation. However, each issue was diagnosed and resolved so that the final Week 9 workflow could run properly. The main issues and their solutions are summarized below:

- **Robot spawned outside the warehouse:** the initial spawn position or world alignment was incorrect. The launch and environment setup were adjusted so the TurtleBot3 started inside the intended warehouse area.
- **Initial map looked incomplete or the robot appeared outside the map:** this happened when the environment had not been fully explored yet or the pose/map alignment was not stable. Continuing SLAM exploration and checking the fixed frame in RViz corrected the issue.
- **move_base not running:** an early test showed that the multi-point script could not connect to the action server. The navigation stack had to be launched correctly before running the navigation script.
- **Waypoint calibration:** coordinates for Home, Shelf A, Shelf B, Pickup and Drop-off were refined using RViz clicked points and testing until the robot could reach each target reliably.
- **UI-to-navigation integration:** the control panel had to wait for the *move_base* server first so that the buttons could send goals successfully.

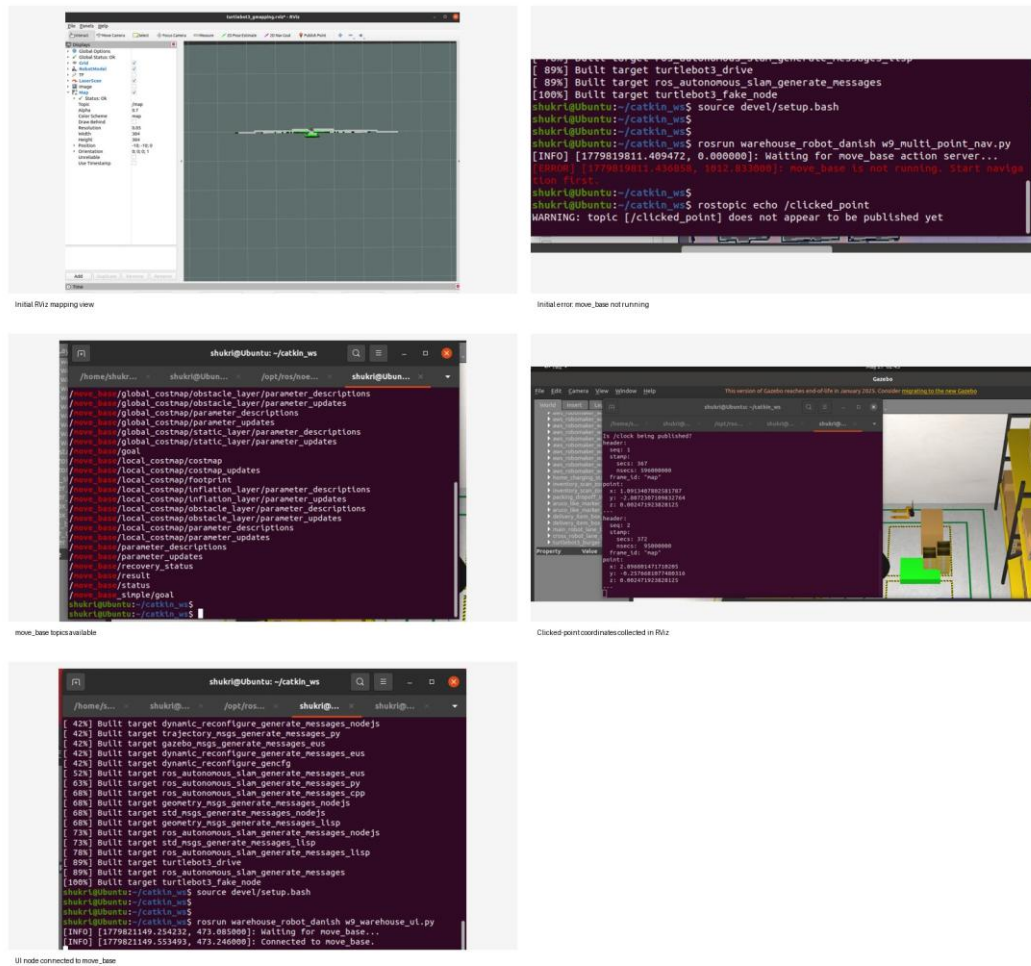


Figure 16. Validation and troubleshooting evidence including the initial mapping view, move_base error, topic checks, waypoint coordinate capture and UI connection confirmation.

9. Additional Navigation and UI Evidence

The following screenshots provide additional proof of the navigation and UI behaviour during testing. They show RViz navigation views, the active control panel window, goal planning behaviour and local costmap updates while the robot was moving through the warehouse map.

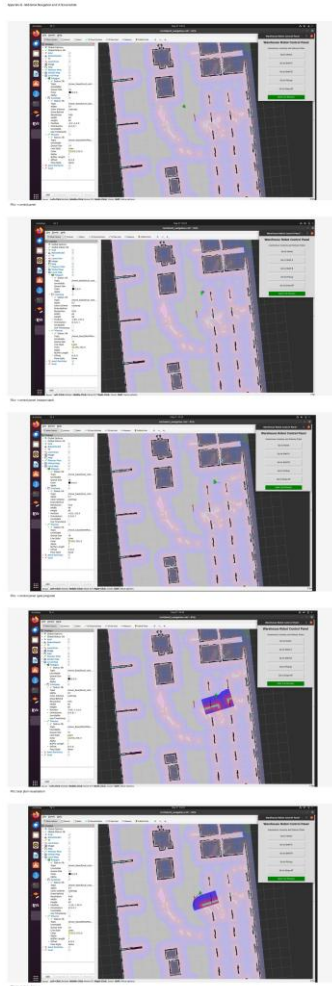


Figure 17. Additional screenshot showing RViz localization, the operator panel and live goal/costmap behaviour during navigation.

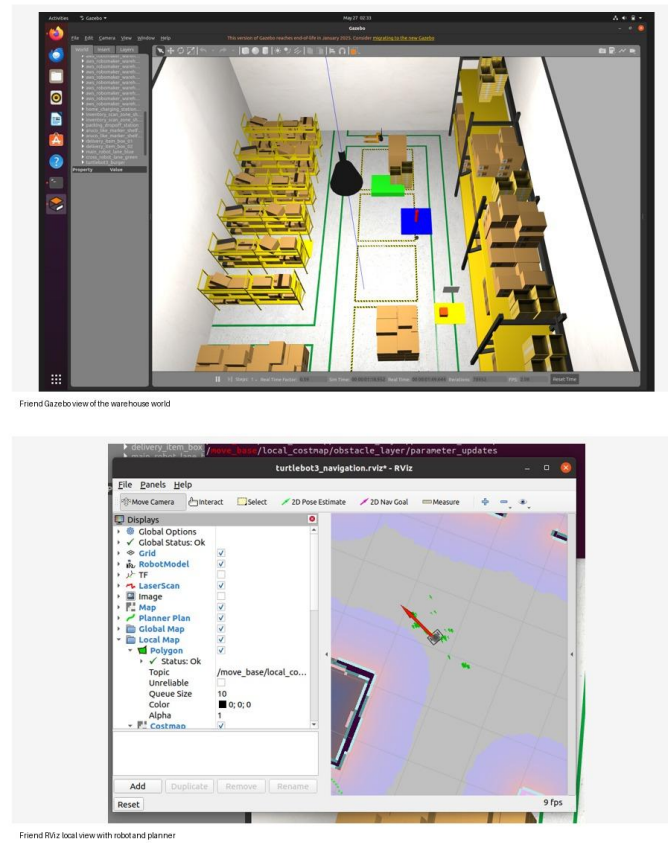


Figure 18. Additional core screenshots showing the Gazebo warehouse scene and a focused RViz planner view.

10. Conclusion

Overall, the Week 9 objectives were successfully completed. We completed the warehouse review, SLAM mapping process and saved-map evidence and multi-point navigation route, waypoint testing and the custom operator interface. Together, these outputs demonstrate that the group has converted the Week 8 customized warehouse world into a functional navigation-ready environment.

This Week 9 result is important because it directly supports the Week 10 task action stage. With the environment built, the map saved, the navigation stack tested and the control panel working, the project is ready for higher-level task execution such as autonomous inventory checking, pickup and delivery routines inside the warehouse simulation.